

Caso Práctico 3

Objetivo y alcance del trabajo

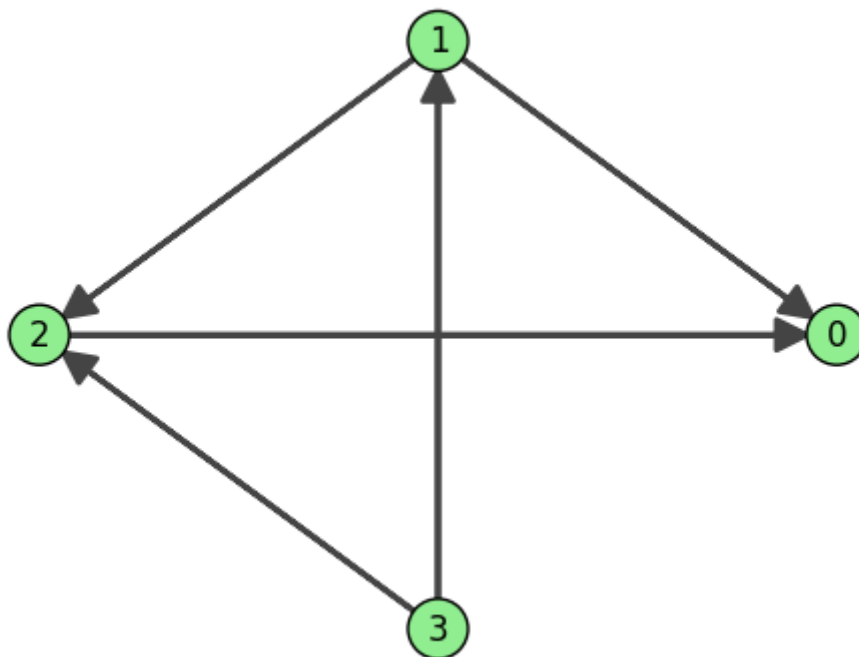
En este tercer caso práctico vamos a seguir utilizando las herramientas del caso práctico anterior para representar grafos y resolver problemas de flujo máximo en Network Flows con la librería `igraph`.

Recuerda:

- Se debe entregar tanto el fichero **.ipynb** como un informe **.pdf**.
- Al puntuar, se valora positivamente contenido extra que aporte valor a la práctica, y es necesario para obtener la máxima nota posible.
- En todos los apartados lo principal es la interpretación que hace el alumno de los resultados, por ejemplo: en caso de que se pida calcular un valor, no limitarse a calcularlo, sino explicar qué significa y qué implica en el contexto de la práctica. Si no se razonan los resultados se valorará de manera reducida.
- El trabajo es grupal, todos los integrantes deben participar para obtener nota. En caso contrario, se debe contactar con el tutor a través de mensaje privado por la plataforma para resolver el problema.

Fase I - Generación y representación de un grafo

El objetivo de esta fase es conseguir **generar el siguiente grafo**:



Utilizando el siguiente código, puedes representar el grafo:

```
import igraph as ig
import matplotlib.pyplot as plt

g = ig.Graph([aristas], directed=True)

fig, ax = plt.subplots()
ig.plot(g, target=ax, layout="circle", vertex_label=range(g.vcount()),
        vertex_color="lightgreen")
plt.show()
```

En el código anterior, la línea:

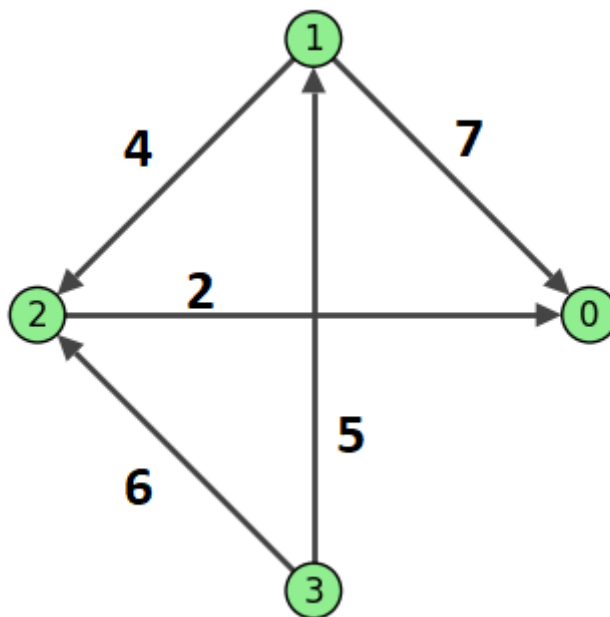
```
g = ig.Graph([aristas], directed=True)
```

Permite añadir las conexiones entre nodos (*aristas*), y especifica que estamos trabajando con un grafo dirigido (*directed=True*).

En caso de querer añadir más conexiones, simplemente añadiremos nuevos elementos. Cambialo para representar el grafo de la imagen anterior.

Fase II - Flujo máximo en el grafo

La librería *igraph* permite resolver problemas de flujo máximo. Se pide **actualizar el grafo generado en la fase anterior para que tenga las siguientes capacidades, y calcular su flujo máximo**:



```
g.es["capacidad"] = [...]
flow = g.maxflow(3, 0, capacity=g.es["capacidad"])
```

Lo primero que debemos hacer es especificar la capacidad de las aristas. Para eso, utilizaremos un array de capacidades (primera línea) las capacidades de las aristas deben incluirse en el mismo orden de las aristas, es decir 6 se corresponde con la primera arista (3,2), 2 es de la segunda (2,0) y así sucesivamente.

En la segunda línea, pedimos que calcule el flujo máximo, teniendo en cuenta que 3 es el nodo fuente, y 0 es el nodo destino del flujo. El ejemplo completo quedaría de la siguiente manera:

```
import igraph as ig
import matplotlib.pyplot as plt

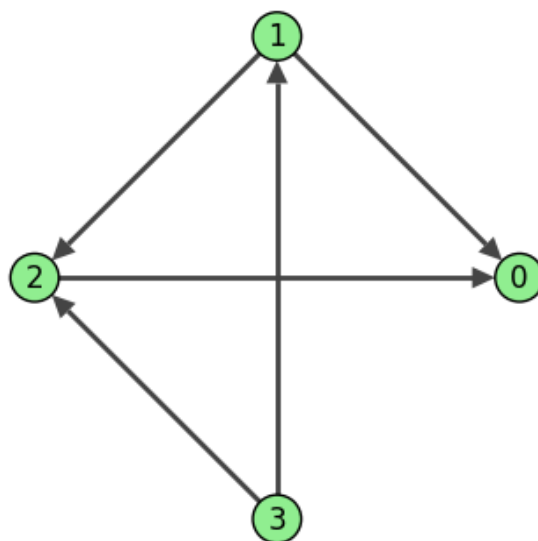
g = ig.Graph([aristas], directed=True)

g.es["capacidad"] = [...]
flow = g.maxflow(3, 0, capacity=g.es["capacidad"])
print("Flujo máximo:", flow.value)
print("Flujo de las aristas:", flow.flow)

fig, ax = plt.subplots()
ig.plot(g, target=ax, layout="circle",
        vertex_label=range(g.vcount()),
        vertex_color="lightgreen")
plt.show()
```

El resultado es el siguiente:

```
Flujo máximo: 7.0
Flujo de las aristas: [2.0, 2.0, 5.0, 0.0, 5.0]
```



Fase III - Aplicación en ejemplo real

Utilizando el código de las fases anteriores para:

- Representar el siguiente grafo con sus capacidades
- Calcular el flujo máximo
- Identificar cuellos de botella

